

EcoVision Security Sentinel

Violence Detection Module — Research Progress Report

Prepared for Thesis/Capstone Advisor Review • August 8, 2026 · Updated August 12, 2026

Module: Real-time violence detection (YOLO person/pose + weapon detection, X3D-XS video classifier)

Abstract

EcoVision Security Sentinel is a real-time video-analytics system for public CCTV, combining a **YOLO-family detector** (person localization, pose keypoints, weapon recognition — a per-frame spatial task) with an **X3D-family video classifier** (violence recognition over a temporal window — a spatiotemporal task). This report documents a full diagnostic and correction pass on the violence-detection subsystem: five distinct defects were found and fixed, raising honest (held-out, never-seen-in-training) test accuracy from 78.4% to 95.0%. A subsequent deployment check against a real, wide-angle city camera surfaced a second, unrelated problem invisible to any benchmark: the model is **blind, not merely less confident**, below roughly 15% of frame height, because training footage never showed it a person that small. An architectural fix (tiled scene inference) and a retrain with scale augmentation were designed, measured, and shipped, recovering city-scale recall from 0/30 to 30/30 on synthetically shrunk clips. Validating the fixed system against **unlabelled, real, continuously-running footage** (35+ minutes of a Davao intersection, plus five additional diverse street-camera clips) surfaced a **third problem, still open**: false-alarm rate on real footage (4–14 alerts/hour depending on scene) remains well above what a human operator could act on, despite being a measured improvement over the prefix baseline (36.0/hour). This is presented honestly as unresolved, together with the specific, measured next steps that would close it — not papered over.

1. Problem Statement

At the start of this work period, the violence-detection component of EcoVision Security Sentinel had been retrained four separate times — varying unfreeze depth, data augmentation, class oversampling, and input representation — without breaking a **held-out accuracy plateau of approximately 70%**. No prior retrain had identified *why* accuracy was capped, only that it was.

The objective of this work was threefold:

1. Diagnose the cause of the plateau through systematic measurement rather than further blind hyperparameter search.
2. Correct whatever defects were found, and re-measure honestly.
3. Verify the corrected model actually works on the system's real intended deployment target — full-view city CCTV — rather than stopping at benchmark accuracy.

Objective 3 surfaced two further, unrelated problems (§5, §12) not visible in any benchmark number, each requiring its own diagnosis and fix.

2. Methods: System Architecture

The system uses **two different convolutional architectures for two different tasks**. This section explains what each is, why each was chosen, and why they cannot substitute for one another — since the two answer structurally different questions about the same video.

2.1 YOLO (You Only Look Once) — spatial detection, per frame

YOLO is a **2D convolutional network**, single-shot object detector: given one image, it predicts bounding boxes and class labels for everything of interest in it, in one forward pass. It answers a purely **spatial** question — *where are the people, and where are the weapons, in this single frame* — and does not, by itself, know anything about what happened in the previous frame.

Two YOLO models are used here, each doing a distinct spatial job:

- **yolo11s-pose** — person detection plus 17 skeletal keypoints per person. Used for (a) locating and tracking people frame-to-frame, and (b) producing person crops for per-track violence classification (§2.5).
- **A separate weapon-detection YOLO model** — trained to recognize weapon classes directly.

YOLO is the right tool here because the task is genuinely per-frame and needs to run at full camera frame rate (30fps) cheaply: 2D convolutions are far less computationally expensive than 3D ones, which is why YOLO can run on every frame while the video classifier (§2.2) only needs to run periodically on a buffered window.

2.2 X3D — spatiotemporal classification, over a clip

X3D is a **3D convolutional network** family (Feichtenhofer et al., *X3D: Expanding Architectures for Efficient Video Recognition*, CVPR 2020) purpose-built for **video** recognition: its convolutions operate over a spatiotemporal volume (height $\times$ width $\times$ **time**), so it can recognize an *action* — something that only exists as a pattern across multiple frames — not just an object present in one frame. It answers a **temporal** question: *given this short window of frames, is what's happening in it violent?*

X3D's specific contribution over earlier 3D-CNN video models (e.g. I3D, SlowFast) is computational efficiency: rather than hand-designing a 3D architecture from scratch, it starts from a small 2D image-classification backbone and **expands it along independent axes** — temporal duration (γt), frame rate (γr), spatial resolution (γs), network width (γw), and depth (γb) — searching for the combination that is accurate per unit of compute, rather than accurate at any cost. This project uses **X3D-XS**, the smallest variant in the family, specifically because the deployment target is a single consumer GPU (GTX 1660 SUPER), not a data-center accelerator.

2.3 Why two different CNN architectures, not one

A single 2D network cannot recognize an action, because an action is defined by *change over time* — a still frame of a person mid-swing looks identical whether the swing is a punch or a wave. A single 3D network, run on every frame at full resolution to do YOLO's job, would be many times more expensive than a 2D detector for a task that doesn't need the temporal axis at all — localizing a person in a frame doesn't require looking backward in time. Using each architecture only where its extra machinery (2D speed vs. 3D temporal reasoning) is actually needed is what makes real-time operation on modest hardware possible at all: **YOLO runs every frame (2D, cheap); X3D runs periodically on a buffered clip (3D, expensive but infrequent).**

2.4 Three operating modes, one shared classifier

The X3D classifier is deployed in three switchable modes (one config edit, `detection.violence.mode` in `config.json`):

Mode	What X3D sees	Best for
track	A YOLO-tracked person's own crop (+ nearby bystanders merged in)	Close/medium-framed cameras where a tracked person occupies a meaningful fraction of the frame
scene	The whole frame, once per check interval	Simple, cheapest whole-frame baseline; blind to small distant people (§5)
tiled	The frame split into an overlapping grid of regions, each independently classified, plus one full-frame pass	Wide, full-view city cameras where a person is too small (6–12% of frame height) for scene mode to see at all — see §5, §10

Every mode shares the exact same confirmation logic (`_smooth_and_confirm` , §2.5) so an alarm always means the same thing regardless of which spatial mode produced it.

2.5 Temporal smoothing and alert confirmation

A raw per-check confidence is not used directly to fire an alert. Each detector (per tile, in tiled mode; once, in scene/track mode) runs an independent state machine:

- 1. **EMA smoothing** (`ema_alpha = 0.35`) damps single-frame noise.
- 2. **Consecutive-hit confirmation** requires the smoothed confidence to clear the threshold for `consecutive_required` checks in a row before an alarm fires — not a single lucky frame.
- 3. **Sticky release with hysteresis**: once confirmed, the alarm stays live until confidence drops a full `release_hysteresis_margin` *below* the threshold, not merely below it, so a boundary-hovering confidence doesn't flicker the alert on and off.

In tiled mode, each tile runs this state machine **independently**, and the combined system alarm is "any tile confirmed" — deliberately, so that one tile catching a real incident and holding it via sticky release is not diluted by other quiet tiles (§10).

3. Datasets

Dataset	Role	Notes
RWF-2000	Training / validation / test (benchmark)	2,000 real-world CCTV-style clips, balanced 50/50 violent/non-violent
SCVD (Smart-City CCTV Violence Detection)	Training / validation / test (benchmark)	Adds normal-class diversity; its "Normal" class scored a perfect 0% FPR in the final held-out test (§4.1), notably cleaner than RWF-2000's equivalent class
Self-recorded Davao/PH live CCTV captures	Real-world qualitative + quantitative validation (unlabelled)	See §12 — 35 min primary intersection feed + 5 diverse clips (barbershop, fiesta, street view, tire shop, traffic), captured directly from public YouTube live streams via <code>yt-dlp / ffmpeg</code> , all stored on D:
CCTV-Fights (NTU ROSE Lab, via Kaggle mirror)	Real-domain training/validation data, acquisition in progress	13.31GB, 1,001 files, ground-truth annotated, genuine CCTV-sourced (as opposed to RWF-2000's mixed sourcing). Kaggle mirror download in progress as of this report (§13); official registration with NTU ROSE Lab for the authoritative source is still pending and is not a same-day task

Why a fourth, real-domain dataset matters: RWF-2000 and SCVD, while described as "real-world," are still curated benchmark clips — median person height 37.1% of frame, balanced class ratios, short duration. None of that matches a continuously-running, full-view city camera, where people are 6–12% of frame height and violence is rare rather than 50% of all footage. This mismatch (**domain shift**) is the direct cause of both major findings in §5 and §12, and is a documented, actively-researched problem in the video-anomaly-detection literature generally, not unique to this system.

All three-way manifest splits (train/val/test) are assigned by **SHA-256 content hash** (§6.2) so that any of these datasets can be added to the pool without risking leakage between splits.

4. Methodology

The guiding principle throughout was **measure, do not assume**. Every claim below is backed by a script and a logged result; several hypotheses formed during the work were subsequently disproven by follow-up measurement and are reported alongside the ones that held, because the negative results are methodologically part of the record.

5. Defects Found and Corrected (Benchmark Accuracy Phase)

5.1 Detection coupled to tracking (the original architectural cause)

The deployed pipeline only classified a person if a single YOLO tracking ID survived 20 consecutive frames. On the held-out set, **132 of 769 clips (17.2%) never reached the classifier at all** — 36 of them violent, silently scored as "normal" because the model never ran, and 96 normal clips scored as true negatives "for free." This inflated the apparent accuracy while hiding the real recall problem.

Fix: a whole-frame ("scene mode") classification path was added (`SceneViolenceDetector` in `x3d_violence_detector.py`), which classifies the entire frame on a fixed interval independent of person tracking. This removed the tracking gate as a source of missed detections.

5.2 Dataset leakage and duplication

An audit of the training data (6,182 source files) found **485 byte-identical duplicate files** and, because the train/validation split was assigned by a random shuffle, **93 held-out clips (12.1%) had an identical byte-for-byte twin in the training set** — the model had memorised, not generalised, on those clips.

Fix: `build_dataset_manifest.py` was written to assign every clip's split **from a SHA-256 content hash** rather than a shuffle. Two files with identical content always land on the same side of the split, making this class of leakage structurally impossible rather than something to re-audit after every dataset change. A **three-way split** (train / val / test) was added specifically so that model-selection (validation) and final reporting (test) could no longer be the same data — see §5.6.

5.3 The double-softmax defect (root cause of the accuracy plateau)

This was the most significant and least visible defect. The underlying model architecture (`pytorchvideo` 's X3D) already terminates its classification head in a `Softmax` layer — its raw output **is already a probability distribution**. Both the training loss (`nn.CrossEntropyLoss` , which internally applies `softmax`) and the live inference code (which applied `torch.softmax` again to the model's output) were **softmaxing an already-softmaxed value**.

Measured consequences:

- **The training loss had a hard mathematical floor.** A perfectly classified example could score no better than `-log(softmax([0,1])[1]) ≈ 0.3133` . Across 390 logged training

batches, the minimum recorded loss was exactly 0.3133, and none went below it — confirming the floor was actually binding, not merely theoretical.

- **This collapsed the loss's dynamic range to just 1.0** (a perfect prediction and a confidently wrong one differed by only 1.0 in loss), which starved the model of gradient signal on its hardest, most informative examples — a strong candidate for the true cause of the four-retrain plateau.
- **Every confidence value the system ever reported, logged, or thresholded was compressed into the range [0.269, 0.731].** A normal scene could never read below 27%, and a violent one could never read above 73%. Every threshold tuned against this compressed scale (e.g., the deployed 0.40) was tuned against a distorted number.

Fix, and a self-correction on the way to it: the first proposed fix — replacing the head's `Softmax` with `Identity` — was tested before being deployed and found to be **incorrect**: the model architecture averages per-timestep probabilities across 10 temporal positions before pooling, so removing the softmax changes what is being averaged (probabilities vs. logits), not merely its scale. This was caught by a controlled test on 120 real clips, which found one verdict that changed at a large decision margin (a logit gap of -1.30) — proof the two formulations are not equivalent. The correct fix — training with `NLLLoss` directly on the model's native probability output, with gradient clipping added as a safety measure once the loss floor was removed — was verified to be **exactly decision-preserving** (0/120 disagreements) at the deployed threshold, while restoring the full $[0, 1]$ confidence range and removing the loss floor entirely.

5.4 Selection bias in reported accuracy

The training script selects its "best" checkpoint by accuracy on the validation set — meaning a validation-set accuracy figure is, by construction, a number the training process already optimized toward. Every accuracy figure reported by this project before this work period was a validation-set figure presented without that caveat. The evaluation script's own output banner asserted such numbers were "the number to cite as true generalization performance," which was corrected to distinguish validation (selection-optimistic) from test (honest) results, and a related bug where test-split results were being mislabeled as validation-split in the permanent log (`eval_history.csv`) was also fixed.

5.5 Supporting fixes

- A checkpoint-resume bug where the "best accuracy so far" value was saved to disk **before** being updated for the current epoch, meaning a training run interrupted and resumed (which occurred once, due to a memory-exhaustion crash during this work period, §11)

would silently forget its actual best score and could overwrite a better model with a worse one. The associated "early stopping patience" counter had the same bug.

- The model's expected input resolution (`frame_size`) was hardcoded identically in two separate files with no mechanism to detect disagreement between them — the exact class of bug that caused §5.1–5.3. A `.meta.json` sidecar is now written next to every trained checkpoint recording its exact input contract (resolution, frame count, output convention), and the live detector reads it and **overrides a mismatched config with a loud warning** rather than silently using the wrong value.

5.6 Honest, leak-free, selection-free evaluation

With the three-way manifest in place, the final model was trained on a 3,298-clip training set, model-selected on a *separate* 722-clip validation set, and — for the first time in this project's history — scored on a **758-clip test set that neither gradient descent nor checkpoint selection ever saw**.

6. Results (Benchmark Accuracy)

Stage	Split type	Accuracy	Recall	Precision	FPR
Start of this work period	selection-optimistic	77.2%*	—	—	—
After leak removal, before softmax fix	honest (leak-excluded)	78.4%	77.0%	91.2%	18.2%
Retrained model, validation split	selection-optimistic	93.9%	96.3%	91.9%	8.4%
Final model, held-out test split	honest — never seen in training or selection	95.0%	97.4%	92.9%	7.4%

*Confusion counts for the 78.4% figure: $TP=237$, $FN=137$, $TN=304$, $FP=91$ ($n=769$). Final honest figure confusion counts: $TP=369$, $FN=10$, $TN=351$, $FP=28$ ($n=758$).

The honest, held-out result represents a **+16.6 percentage point accuracy improvement and a +20.4 point recall improvement** over the session's starting point, achieved on a model trained with **18% less data** than an intermediate checkpoint, which underscores that the gain came from removing defects rather than from more data.

6.1 Error analysis on the held-out result

Breaking the 32 false positives down by data source found **all 32 originated from one subclass** (RWF-2000's "NonFight," largely crowded/wide street footage); the parallel SCVD "Normal" class scored a perfect 0% FPR on 171 clips. Further analysis found false-positive rate is strongly correlated with **clip duration** — 0% at 1–5 inference checks per clip, rising to 19.9% at 10+ checks — meaning **per-clip false-positive rate systematically understates the false-alarm rate on a continuously running camera**, which never stops accumulating checks. This directly foreshadows §12.

7. A Deployment-Blocking Discovery: Scale Mismatch on Wide Camera Views

Following model correction, the corrected system was tested against a real, unlabelled public CCTV stream (a street-crossing camera in Davao City) as a qualitative deployment check. Initial observation: confidence readings fluctuated with traffic on a wide-angle view of the intersection, with no violence occurring.

Working hypothesis (formed, then tested, then disproven): it was hypothesized this fluctuation was the model misreading vehicle/crowd motion as violence (a false-positive risk). This was tested directly by taking the same live footage and *synthetically shrinking* the people in it while holding everything else constant. **Confidence did not rise — it barely moved (0.132 → 0.128), and no alarm fired at either scale.** The false-positive hypothesis was rejected by this test.

The actual mechanism, confirmed by controlled measurement: every training clip was measured for person size as a percentage of frame height using the existing pose-detection model. Training footage puts a person at a median of **37.1%** of frame height (range 24–60%). The Davao street camera, uncropped, puts a person at roughly **9–12%**. To determine the effect of this gap directly, 40 violent clips the model detects with 100% confidence in their original form were replayed at progressively smaller synthetic scale:

Person height (% of frame)	37%	30%	22%	19%	15%	9%
Clips still detected (of 40)	40/40	34/40	25/40	22/40	7/40	0/40

There is a sharp cliff between 19% and 15% person-height. **Below approximately 15%, the model does not become less confident — it detects nothing.** This is the correct explanation for the earlier field observation: a wide-angle street camera does not cause false

alarms; it causes the system to be **effectively blind**, and a blind camera on a quiet street is statistically indistinguishable from a working camera on a quiet street. For a public-safety system, this is the most dangerous possible failure mode, because it produces no symptom.

7.1 Why the obvious fix (cropping the camera view) was rejected

The natural first response — configure each camera to crop/zoom into the region where people are expected — was proposed and then explicitly rejected after review, on the following reasoning: a camera is installed to monitor a specific area; cropping it to compensate for a model limitation reintroduces exactly the blind spot the camera was purchased to eliminate. This would violate the system's core purpose for the sake of a training-data artifact.

7.2 Root cause identified in the training pipeline

Investigating why the model had never learned to recognize small people, the clip-augmentation code was found to contain a **scale augmentation that could only zoom in** (random crop 80–100%, resized back up — which only ever makes a person *larger* than in the source footage). **No augmentation in the pipeline had ever shown the model a person smaller than what the raw source footage happened to contain.** This is now understood as the root cause of the scale-blindness finding, not merely a symptom of it.

7.3 Coverage-preserving architectural alternative

A tiled inference approach was designed and implemented as an alternative to cropping: the frame is divided into a grid of overlapping regions (each independently classified with its own temporal buffer), so a distant person becomes proportionally larger within their tile **without discarding any part of the camera's field of view**. A controlled test replaying the same shrunk clips through this design recovered **25 of 30** detections that a whole-frame pass missed entirely at the same scale (0 of 30), while covering 100% of the frame. A subtlety was found and addressed during this test: a non-overlapping grid can split a single incident across a tile boundary and cause it to be missed by every tile (measured: a non-overlapping 2×2 grid scored *worse*, at 1/30, than the whole-frame baseline) — solved by overlapping tiles plus retaining one full-frame pass for scene-scale context.

8. Related Work

- **CUE-Net** (CVPR 2024 Workshop, RWF-2000 SOTA) handles distant/partially-obscured subjects via **spatial cropping around detected people** — the state of the art solves the

same scale problem this project found, by cropping to people rather than classifying whole wide frames. This project's `_crop_person()` already implements a CUE-Net-style crop (60% padding, bystander merge), but §10 (T1) found the *prerequisite* — reliably detecting a person at city-camera scale in the first place — fails before the classifier ever runs, which CUE-Net's own benchmark conditions do not require it to face.

- **Skeleton/ST-GCN**-family action recognition is scale-invariant by construction (it operates on pose keypoints, not pixels), and this system already computes those keypoints for its per-track mode — a plausible future direction, gated on the same pose-detection reliability problem at distance (§10, T1).
- **Domain shift between benchmark and deployment CCTV footage** is a documented, actively-researched problem in the video-anomaly-detection literature generally, not unique to this system — this project's own §12 false-alarm-rate gap is a direct, measured instance of it.
- **Weakly-supervised video anomaly detection** (UCF-Crime, MIL-ranking approaches) frames the problem differently: rather than a binary per-clip classifier trained on balanced benchmark clips, these methods train directly on continuous, realistically-imbalanced footage (violence as a rare event, not 50% of the data) — closer to this system's actual deployment condition than RWF-2000/SCVD are, and a candidate direction for future work once enough real-domain footage (§3) exists to train on.
- **CCTV-Fights** (NTU ROSE Lab) is, to date, the closest publicly available dataset to this project's actual deployment condition: genuine CCTV-sourced footage with ground-truth annotation, rather than a curated, balanced benchmark. Acquiring it is in progress (§13).

9. Phase 0 Results: the Architecture Question, Settled by Measurement

The plan following §7 was to decide — before writing any new training code — whether distant, city-scale people should be handled by cropping to a detected person (the CUE-Net-style approach already partially built into this codebase) or by the coverage-preserving tiled approach of §7.3. Three measurements were run to answer this rather than assume it.

T1 — can the existing pose detector even find a city-scale person? The pose model was swept across four input resolutions against people synthetically shrunk to the Davao camera's measured scale (9% of frame height). Result: **the gate failed**. At an affordable resolution (1280px, 30ms/frame), the pose model located the person in only 32.5% of frames. Pushed to the most expensive resolution tested (1920px, 291ms/frame — far past any real-time budget), detection still reached only 46.7%. Person-cropped classification cannot work if the person is never reliably detected in the first place, independent of how good the classifier afterward is.

T2 — head-to-head confirmation on the same shrunk footage. Whole-frame, tiled (§7.3), and per-track (pose + crop) modes were run on the same 30 held-out violent clips shrunk to 9% person-height:

mode	clips detected (of 30)
whole-frame	0/30
per-track (pose + crop)	2/30 — pose located a person in only 38.0% of frames
tiled (4×4, 25% overlap, + full-frame pass)	22/30

This corroborates T1 directly: per-track's own architecture is scale-invariant once a crop is obtained, so its near-total failure here traces back to the pose stage failing to find anyone to crop, not to the classifier.

T3 — cost per camera. Real-time capacity requires clearing a fixed budget of 33.33ms per frame (the true frame-arrival period at 30fps — the live pipeline calls its detector on *every* frame, not on a longer "check window," which an initial pass at this measurement mistakenly divided by instead, inflating every capacity figure by roughly 15×; this was caught by re-reading the calling code and corrected before being reported here). Measured against the correct budget:

mode	ms/frame (busy)	cameras sustainable per GPU
whole-frame	3.31	~10
tiled 4×4 (as first implemented)	49.89	0.67 — cannot sustain even one live camera
per-track (pose + crop)	103.96	0.32 — cannot sustain even one live camera

Tiled mode won T1/T2 on capability but, as first implemented, failed T3 on cost — 17 tile inferences were being run as 17 separate sequential GPU calls per check. This was diagnosed (isolated to the per-call dispatch overhead, not the per-frame buffer maintenance) and fixed by batching all 17 tiles into a single GPU call: an isolated compute-only test measured a consistent ~1.9–2.7× speedup from batching depending on run. Projected against that isolated number alone, this looked sufficient to clear the real-time budget — **that projection was reported prematurely and has since been corrected.**

Correction, reported for the same reason every other self-correction in this document is: the isolated speedup test measured GPU compute only, using a pre-made tensor already resident on the GPU. It did not include the cost of building a real 17-tile batch from live video (cropping, resizing, normalizing, and transferring host→device), and — more materially — every full-pipeline measurement taken up to that point was run while a second process (the system's own live dev server, or the measurement chain's own later steps) was concurrently

holding the GPU, which was not caught until each earlier "clean" figure was checked against the live process list rather than assumed. With every other GPU consumer confirmed stopped, the honest full-pipeline number is:

mode	ms/frame (clean, GPU confirmed idle)	cameras sustainable per GPU
whole-frame	1.92	~17
tiled 4×4, interval=15 (batched, as first implemented)	56.76	0.59 — cannot sustain one live camera
per-track (pose + crop)	~38.9	~0.86

Batching the 17 tile calls is a real, measured improvement in raw GPU compute, but on its own it did not close the gap — tiled mode's full-pipeline cost remained under the real-time budget by roughly half.

Repair, tested directly rather than assumed: two further levers were tried on the same clean, GPU-idle protocol. Reducing the grid from 4×4 (17 tiles) to 3×3 (10 tiles) — trading some overlap margin for ~40% fewer inferences per check — cost exactly one clip out of the 30-clip city-scale recall benchmark (30/30 → 29/30) while dropping cost to 34.34 ms/frame (0.97×, still short). Combining that with a longer check cadence (15 → 20 frames, i.e. checking every ~0.67s instead of every 0.5s at 30fps) reached **28.48 ms/frame — 1.17× real-time capacity — with recall unchanged at 29/30**. This clears the budget with margin and is now the shipped default (`tile_grid: 3` , `tile_check_interval: 20` in `config.json`).

Decision, now actionable: tiled whole-frame scene mode (3×3 grid, 25% overlap, plus one full-frame pass, batched inference, 20-frame check interval) is the architecture for full-view city cameras — the only one of the three that reliably detects at city-camera scale (T1/T2), and now the only one that also clears the real-time throughput budget. It is wired into `config.json / main.py` as `"mode": "tiled"` , switchable back to `"scene"` or `"track"` by a single config edit, per the requirement that every mode stay reversible. Person-cropped/two-stage detection remains set aside for city cameras on both original grounds (pose cannot reliably find distant people; its own cost also falls short of one real-time camera).

A stale-dependency issue found alongside this work is also fixed: the pose and weapon detectors' `.engine` (TensorRT) files were failing to load ("platform specific tag mismatch" — a TensorRT engine is tied to the exact runtime/driver version it was compiled on, and these predated the current install) and silently falling back to slower PyTorch weights on every run. Both were rebuilt on this machine via the project's existing `optimize_weights.py` and verified to load and run inference successfully.

10. Phase 1 & 2 Results: Retrain Complete, Honest Numbers In

The whole-frame model was retrained on the same leak-free three-way manifest (§5.6), with the scale-augmentation range widened specifically to close the blind zone identified in §7: the augmentation now shrinks training clips down to as little as 6.7% person-height — below the measured 9% blind floor, with margin — so the model was exposed to city-scale people during training for the first time, not only at inference. Training completed (early stopping, 15 epochs, best validation accuracy 92.9% — a validation-set figure only, selection-optimistic per §5.4).

City-scale recall — the entire point of this retrain — improved dramatically, replayed on the same 30 clips shrunk to 9% person-height used throughout Phase 0:

mode	old weights (baseline)	new (scale-augmented) weights
whole-frame	0/30	28/30
tiled	22/30	30/30

Honest test-split accuracy, however, regressed against the §6 baseline:

Metric	old (deployed) model	new (scale-augmented) model
Accuracy	95.0%	92.5%
Recall	97.4%	96.3%
Precision	92.9%	89.5%
False Positive Rate	7.4%	11.3%

This does not recover with threshold tuning. A full sweep against the same test-split confidences found no threshold that matches the old model on both axes: at the default 0.50 the new model's recall is slightly better (97.6%) but FPR is nearly double (12.4%); pushed as high as 0.95, FPR drops below the old model's (4.5%) but recall falls to 91.8% and accuracy still caps at 93.7%, short of the 95.0% baseline at every threshold tested. **This is a genuine capability trade-off from widening the scale-augmentation range, not a tuning artifact** — the model that can see a person at 9% of frame height pays for that with reduced discrimination at ordinary, close-up scale.

One operational incident occurred during training and is recorded for completeness: partway through, the host machine ran out of committable system memory (unrelated concurrent processes, not a leak in the training code) and the training process crashed. Recovery used the run's own checkpoint/resume mechanism, which restores epoch number, best-validation-

accuracy, and early-stopping-patience state rather than resetting them — a bug in exactly this mechanism was one of the fixes reported in §5.5, and it held up correctly here on its first real use. No progress was lost.

A second, unrelated operational lesson came from the cost re-measurement in §9: the automated evaluation chain checked that the training process had exited and that free memory was adequate before calling its own numbers "clean," but had no way to know the system's own live dev server was independently running and holding the GPU the whole time. "GPU idle" turned out to need an explicit process check, not an inference from "nothing I started is running." This is now handled by checking the live process list directly rather than assuming it.

11. Operational Note: Two Unrelated System Incidents

For completeness, this section records two incidents outside the modeling work itself:

1. **Training crash (§10)** — resolved via the existing checkpoint/resume mechanism; no data or progress lost.
 2. **A second machine incident during this same work period:** while downloading the CCTV-Fights dataset (§13) to expand the real-domain training pool, the C: system drive was driven to **0 bytes free**, because the download library's default cache directory (`C:\Users\User\.cache\kagglehub`) is not on this machine's designated project/data drive (D:). The download crashed mid-extraction, leaving a 17.2GB corrupt partial archive. This was diagnosed, the corrupt cache was removed (with explicit confirmation before any deletion, since it touched a location outside the project directory), and the download was relaunched with its cache directory explicitly redirected to D: via the library's `KAGGLEHUB_CACHE` environment variable — the correct, permanent fix rather than a one-time cleanup.
-

12. Phase 3 Results: Real-World Validation — a Third, Still-Open Problem

Sections 9–10 fixed detectability (a blind camera now sees people) and re-confirmed benchmark accuracy stayed close to the original baseline. Neither measures the thing that actually determines whether a human operator can use this system: **how often does it cry wolf on ordinary footage with no violence in it?** §6.1 already flagged that per-clip benchmark FPR

structurally understates this, because a continuously-running camera never stops accumulating inference checks the way a 5–10 second benchmark clip does. Phase 3 measures it directly, on real, unlabelled footage.

12.1 Primary measurement: 35 minutes of continuous Davao intersection footage

Three configurations were run head-to-head on the identical footage, to separate two variables the earlier phases had left confounded (model weights vs. tiled architecture):

Configuration	Alarms / hour
1. Old weights, scene mode (previously deployed)	36.00
2. New (scale-aug) weights, scene mode	296.57
3. New (scale-aug) weights, tiled mode	32.57

Configuration 2 is uniquely bad: the scale-augmented model, run in whole-frame scene mode on a genuinely wide view, produces an order-of-magnitude worse false-alarm rate than either alternative — a scale-sensitivity artifact interacting badly with a wide field of view it was never architecturally suited to see as one frame. Configuration 3 (new weights + tiled architecture together) is a **modest net improvement** over the original deployed baseline (32.57 vs 36.00/hour), while also being the only configuration with proven city-scale recall (§10). This head-to-head is why **new weights + tiled mode**, not new weights + scene mode, is the system's current leading candidate (§14).

12.2 Offline operating-point sweep

Because the exact confirmation logic (§2.5) is a pure function, raw per-check confidence traces were captured once from the same footage and then replayed offline across a grid of threshold × consecutive-required combinations, without re-running any GPU inference — letting many operating points be tested cheaply against real footage rather than only the RWF-2000/SCVD benchmark. One counterintuitive finding: **for tiled mode specifically, raising the threshold or the consecutive-hit requirement made the false-alarm rate worse, not better**. The mechanism, confirmed rather than assumed: overlapping tiles cause real motion (e.g. a person crossing the frame) to sweep sequentially across several tiles. A **low** bar lets the first tile that sees it confirm and hold the alarm via sticky release — one long alarm. A **high** bar causes several tiles to each partially climb toward confirmation and fall back as the motion moves on — several short, flickering alarms, a worse outcome by the "alarms per hour" metric even though each individual tile is being more conservative. The system's current default

operating point (threshold 0.50, consecutive=1) was confirmed by this sweep to already be at or near the best available point for tiled mode on this footage — tuning alone does not resolve the problem in §12.3.

12.3 Diverse-footage validation

To check whether the primary-footage result (§12.1) generalizes, the leading candidate (new weights, tiled mode, default operating point) was additionally run against five independent real street-camera clips, captured from different locations and scene types:

Clip	Scene type	Duration	Alarms/hour
barbershop	quiet storefront	15.0 min	4.00
fiesta	crowded festival	8.7 min	13.81
streetview1	street view	15.0 min	8.00
tireshop	quiet storefront	15.0 min	4.00
traffic	street traffic	15.0 min	4.00
Weighted average (68.7 min total)			6.11

All five clips complete. The pattern is coherent with what the system should do: a genuinely busy, crowded scene (fiesta) runs hotter than quiet storefronts and ordinary traffic (which cluster tightly at 4.00/hour), a sensible response to real activity level rather than random noise. But even the quietest, most favorable scenes still produce one false alert every 15 minutes on footage containing no violence at all — consistent with, not an exception to, the primary-footage finding (§12.1).

12.4 Honest assessment: this is not yet a usable false-alarm rate

Commercial video-analytics deployments generally target under roughly one false alarm per camera **per day** to remain usable by a human operator; below that threshold, operators learn to ignore the system, defeating its purpose. The measured rates here (4–14 alarms per **hour**) are roughly two orders of magnitude above that bar. This is presented plainly rather than minimized: **the recall fix (§10) and the false-alarm-rate fix are two different, only partially related problems, and only the first is solved.**

The most likely root cause is the same one identified in §3/§7: every number in this project, including the retrain in §10, is still trained and validated on **RWF-2000 and SCVD** — curated, balanced benchmark clips — never on ordinary, real, continuously-running street footage where "nothing happening" is the overwhelming majority class. The model has still never been shown

what boring, ordinary motion (traffic, pedestrians, wind, lighting change) looks like at the scale and framing of a real deployment camera. This is precisely the gap the CCTV-Fights acquisition (§13) and the ongoing live-footage capture (§3) are intended to close, though closing it requires an actual retrain on that data, which had not yet occurred as of this report (§14).

13. Real-Domain Data Acquisition (In Progress)

Two efforts are underway to close the domain gap identified in §12.4, rather than continuing to tune around it:

1. **CCTV-Fights (Kaggle mirror)** — a 13GB, ground-truth-annotated dataset of genuine CCTV-sourced fight and non-fight footage (NTU ROSE Lab). Download completed and was verified on this machine (redirected to D:, §11): `ground-truth.json` parses cleanly (ActivityNet-style format — `duration`, `subset`, `frame_rate`, and per-clip `annotations` giving the exact `[start_sec, end_sec]` temporal segment of each fight, not just a clip-level label), with **1,000 annotated entries** exactly matching the file counts on disk (`CCTV_DATA` : 140 training / 70 testing / 70 validation = 280 genuine-CCTV clips; `NON_CCTV_DATA` : 360 / 180 / 180 = 720 non-CCTV clips). This is now a training-usable, non-benchmark, real-CCTV data source available to this project — not yet used in a retrain (§15).
2. **Official NTU ROSE Lab registration** — the Kaggle mirror is unofficial; the authoritative source requires account registration and a Release Agreement at the ROSE Lab's own site. This is **not started**, and realistically is not a same-day task — noted here as necessary future work, not a gap in this report.
3. **Continued opportunistic live-footage capture** — the same six Davao/Philippine public CCTV YouTube streams used for §12 continue to be sampled periodically. This can only ever produce real **negative** examples (no violence is expected to occur on camera during ordinary capture windows) — a structural limitation worth stating plainly: real-domain *positive* (violent) examples can only come from an annotated dataset like CCTV-Fights, not from passive live capture.

14. Current Deployment Decision

As of this report, the system is configured to run **new (scale-augmented) weights in tiled mode** (`config.json` : `"mode": "tiled"`, `scene_model_path` pointing at the scale-augmented checkpoint) — the configuration validated in §12.1 as a net improvement over the original baseline on real footage, and the only configuration with proven city-scale recall (§10). This is

presented as the current best-evidenced choice, **not as a finished, production-ready result**: §12.4's false-alarm-rate finding applies fully to this exact configuration. The previous configuration (old weights, scene mode) remains available by a one-line edit to `config.json` and is fully reversible.

15. Remaining Work

1. **Close the domain gap (§12.4, §13)** — this is now the critical path, not an optional follow-on. CCTV-Fights (1,000 real, annotated clips) is now downloaded and verified (§13); still needed: (a) accumulating enough real-domain negative footage from live capture, (b) an actual retrain/fine-tune on the combined pool, not merely threshold tuning on the existing model.
2. Investigate whether **person-crop mode** produces fewer false alarms on empty/quiet scenes specifically — set aside earlier under confounded measurement conditions (§9, T1/T2), worth a clean, isolated rerun now that the false-alarm problem is the dominant open issue.
3. Complete official NTU ROSE Lab registration for the authoritative CCTV-Fights source (§13), for redistribution/citation legitimacy beyond the Kaggle mirror.
4. **Phase 5** — document the final cameras-per-GPU scaling path with the now-clean, resolved tiled-mode figures (§9), and where edge devices would fit for cameras beyond a single GPU's capacity.
5. Consider a milder scale-augmentation ablation (between the original zoom-in-only pipeline and the current wide range) as a way to recover some of §10's accuracy regression without giving back the city-scale capability gain — untested, and a larger retrain effort than anything above.

The person-cropped retrain set aside by the §9 decision remains a secondary, lower-priority item for city-wide cameras specifically — useful for close-framed cameras, not the city-wide critical path.

16. Limitations to State Plainly

- All benchmark accuracy figures in this report (§6, §10) are measured on **RWF-2000 and SCVD** — not on Philippine street-camera footage. This is the direct, identified cause of §12's false-alarm-rate finding, not a separate unrelated caveat.

- **False-alarm rate on real, continuously-running footage (4–14/hour) is not yet at a usable operating point** for an unattended public-safety alerting system, despite being a measured net improvement over the pre-fix baseline. This is the single most important open problem in the project as of this report.
- Real-camera **recall** (does it actually catch a real incident on a real deployment camera) remains formally unmeasured, because none of the real footage captured so far contains genuine violence — a structural limitation of passive live capture (§13) that only an annotated real-domain dataset (CCTV-Fights) or a live incident can resolve.
- Two other detection capabilities in this system — robbery and vandalism — are implemented as hand-written geometric rules rather than trained models, and report fixed, hardcoded confidence values to the dashboard rather than measured ones. They remain out of scope for this work period (violence detection only, by explicit decision) and are noted here as a known gap for future work.

This report was compiled from the session's logged experimental results (`eval_history.csv`, `train_log.csv`) and the measurement scripts referenced throughout, all retained in the project repository for reproducibility.